Sourceline

Enterprise Readiness

27 October 2026

Contents

1. Assessment at a glance 2
2. Data protection and recovery — the priority 2
3. Application security 3
4. Engineering practice 4
5. Observability 4
6. Capacity and scaling 5
7. Operations and support 5
8. Data governance and compliance 6
9. Functional gaps that matter at group scale 6
10. What to do, in order 7
11. A fair summary 8

What it would take for Sourceline to be run as enterprise software: a system a group can depend on for financial records, defend to an auditor, and hand to a different team without losing the ability to change it safely.

This is an honest assessment, not a sales document. The application works and the procurement controls inside it are genuinely good. What is thin is everything around it — the parts that matter on the day something goes wrong rather than the day it is demonstrated.

1. Assessment at a glance

Area	Where it stands	Gap
Procurement controls	Strong — enforced in the path of the work	None material
Separation between business units	Strong — verified, fails closed	None material
Application security	Partial — authentication is sound, two-factor is off, dependencies are stale	Significant
Data protection and recovery	**Weak — backups unconfirmed, likely none**	Critical
Engineering practice	Weak — no automated tests, no CI, manual deploys	Significant
Observability	Weak — no monitoring, errors surface only to users	Significant
Capacity and scaling	At risk — the cluster is near a hard ceiling	Significant
Operations and support	Absent — no runbooks, no on-call, no incident process	Significant
Governance and compliance	Partial — audit trail is good, data policy undefined	Moderate

Two items are in a different category from the rest. Everything else is work; **the absence of a confirmed backup is a risk being carried right now**, and it grows with every real transaction recorded.

2. Data protection and recovery — the priority

The database runs on a tier whose 500-collection ceiling identifies it as an Atlas shared tier. Those tiers carry no automated backup. The system is about to hold purchase orders, invoices and payment records.

What is needed

#	Requirement
2.1	Confirm the current tier and backup arrangement in writing

#	Requirement
2.2	Automated daily backups with a defined retention period
2.3	Point-in-time recovery, so a loss can be undone to the minute rather than the day
2.4	A stated recovery point objective — how much data the group accepts losing
2.5	A stated recovery time objective — how long a restore may take
2.6	A restore actually performed into a scratch environment, and timed
2.7	That restore repeated on a schedule, because an untested backup is a belief, not a control

2.6 is the one that gets skipped. A backup nobody has restored is not known to work.

Also needed: fifteen separate databases mean a restore may be needed for one unit without disturbing the other fourteen. That should be tested specifically, because it is the likely real case.

3. Application security

Sound today. Passwords are hashed with bcrypt. Sessions are JWTs bound to their business unit and refused elsewhere. Authorisation is enforced per role on the server. CORS is driven by the registry rather than a static list. An audit log records actions and survives a transaction reset.

Gaps

#	Gap	Why it matters
3.1	Two-factor sign-in is switched off in production	Knowing an email address is enough to sign in as that person, finance included
3.2	28 known dependency vulnerabilities — 2 critical, 19 high	Unpatched known issues in code that handles money
3.3	No dependency scanning in the pipeline	The count will grow silently
3.4	No penetration test	Nobody hostile has examined it
3.5	Secrets live in Vercel project settings only	No rotation schedule, no audit of who has read them
3.6	No rate limiting on sign-in	Password and code guessing are unbounded
3.7	No session revocation	A token stays valid until it expires, even after a dismissal
3.8	Password policy is minimal	No complexity floor, no expiry, no reuse check
3.9	No formal access review	Nobody periodically confirms who still needs access

3.1 and 3.2 are the two to fix first. The rest are ordinary hardening.

4. Engineering practice

This is the area that most determines whether the system can still be changed safely in a year.

#	Gap	Consequence
4.1	No unit or integration test suite	Every change is verified by hand, or not at all
4.2	No continuous integration	Nothing stops a broken commit reaching the main branch
4.3	Deploys are manual CLI commands	A deploy depends on one person running the right command from the right directory
4.4	No git-connected deployment	No automatic record of which commit is in production
4.5	No staging parity	Staging was misconfigured for weeks without anyone noticing
4.6	No code review process in evidence	Work merges to the main branch directly
4.7	No release versioning or changelog	"Which version are you on?" has no answer
4.8	No rollback procedure	Recovery from a bad deploy is improvised

What exists and is worth keeping. A fifty-step end-to-end suite that runs against a live API, an isolation check that proves units cannot see each other, and a numbering check that proves concurrent documents get distinct numbers. These are good tests. They are not run automatically, and they are the whole of the automated verification.

What to build, in order

1. Continuous integration running typecheck, the isolation check and the numbering check on every push.
2. Unit tests for the rules that carry money: the three-way match, the approval chain, supplier eligibility, the quotation minimum and its waivers. These are pure logic and cheap to test.
3. Git-connected deploys, so production always corresponds to a known commit.
4. The end-to-end suite running nightly against staging.
5. A documented rollback, tested once.

The order matters. Continuous integration first makes every later change safer; tests written after a CI pipeline exists get run.

5. Observability

Today, the first indication that something is broken is a user saying so. The dashboard defect found during the smoke test had been failing in production for an unknown period, for one role, with nobody aware.

#	Requirement
5.1	Error tracking that captures server exceptions with context and alerts a human
5.2	Uptime monitoring on each unit's address and on the API
5.3	Structured logs, retained, searchable by business unit and request
5.4	Alerting with a defined destination and an owner
5.5	Performance visibility — dashboards currently take six to seven seconds on a cold start, and nobody is measuring whether that worsens
5.6	A health endpoint checked continuously rather than on request

The in-app alert jobs — low stock, RFQ deadlines, document expiry, supplier re-evaluation — are business alerts, not operational ones. They tell a buyer something; they tell nobody that the system is down.

6. Capacity and scaling

#	Issue	Detail
6.1	The cluster is near a hard ceiling	471 of 500 collections used; production alone accounts for about 334 permanently
6.2	Development and staging share it	When the ceiling is reached, writes fail everywhere, production included
6.3	Serverless cold starts	A unit not used for a while responds slowly on first use
6.4	Connection pooling across fifteen databases	Works today; untested under concurrent load across many units
6.5	No load testing	Nobody knows the behaviour at fifteen units of real traffic
6.6	No per-unit resource isolation	A heavy report in one unit is served by the same functions as every other

6.1 and 6.2 together are the practical risk: a ceiling reached in development stops purchasing in production.

7. Operations and support

Nothing in this section exists yet. For a system handling group procurement it is the most conspicuous absence after backups.

#	Requirement
7.1	Runbooks for the operations that are currently tribal knowledge: adding a unit, resetting a unit, restoring data, rotating a secret, rolling back

#	Requirement
7.2	A named owner for the system, and a deputy
7.3	A support route for users, with expected response times
7.4	An incident process: who is called, who decides, who informs the business
7.5	A change process: who may deploy, when, and with whose approval
7.6	Maintenance windows, communicated
7.7	Scheduled access reviews

Most of this is writing rather than building. It is also what makes the difference between software one person maintains and a system the group owns.

8. Data governance and compliance

#	Requirement	Status
8.1	Audit trail of significant actions	Exists, and survives a transaction reset
8.2	Retention policy — how long records are kept	Undefined
8.3	Personal data inventory — the system holds supplier contacts and bank details	Undefined
8.4	Compliance with Zimbabwe's data protection law, and South Africa's for Phase 4 units	Not assessed
8.5	Encryption in transit	Yes, HTTPS throughout
8.6	Encryption at rest	Depends on the Atlas tier; to confirm with 2.1
8.7	Segregation of duties	Enforced in the approval chain; not reviewed against the written procedure
8.8	Evidence pack for an auditor	Does not exist

8.7 deserves attention. The system enforces a four-stage chain and a USD 5,000 threshold because that is how it was built. Whether those match each unit's actual delegated authority has not been checked, and the system will enforce the wrong rule just as faithfully as the right one.

8.3 is not hypothetical: supplier bank details are held, and bank-change control exists precisely because they are a fraud target.

9. Functional gaps that matter at group scale

These are known and deliberate, but each has a business consequence worth naming.

Gap	Consequence
No group-wide reporting	No single view of what the group spends, which was a main reason for one platform
No shared supplier master	A supplier is onboarded, assessed and evaluated separately in every unit it trades with
No currency on orders, invoices or payments	The USD 5,000 threshold is applied to non-USD values as though they were USD. Blocks the South African units
No per-unit branding on documents	Every unit's purchase order prints another company's name
No per-unit approval thresholds	One limit for a cement producer and a retailer alike
Document numbers repeat across units	A supplier serving several units receives different orders with the same number
No framework contracts or price comparison	The synergy case for a group platform is not yet realised

The first three have the most weight. The third also blocks Phase 4 of the rollout.

10. What to do, in order

Immediately — risk being carried now

1. Confirm the backup position and fix it if absent. Test a restore.
2. Turn two-factor sign-in back on, once administrators have real addresses.
3. Move development and staging off the production cluster.
4. Triage the 2 critical and 19 high dependency vulnerabilities.

Before the group depends on it

5. Continuous integration: typecheck plus the three existing checks, on every push.
6. Error tracking and uptime monitoring, alerting a named person.
7. Unit tests for the money rules — three-way match, approval chain, eligibility, quotation minimum.
8. Git-connected deploys and a tested rollback.
9. Runbooks for the operations that are currently in one person's head.
10. A named owner, a support route and an incident process.

Before it can be called enterprise

11. Retention and personal-data policy; a data protection assessment for both countries.
12. A penetration test, and remediation.
13. Secret rotation, rate limiting on sign-in, session revocation.
14. Load testing at fifteen units of realistic traffic.

15 Verify each unit's approval chain against its written delegation of authority.

16 An auditor's evidence pack.

To realise the group case

17 Currency on all money documents — also unblocks the South African units.

18 Group-wide reporting.

19 Per-unit branding, numbering and thresholds.

20 Framework contracts and cross-unit price comparison.

11. A fair summary

The procurement logic is the hard part, and it is done and demonstrably correct. Items 1 to 10 are mostly configuration, pipeline and writing rather than invention — a few weeks of focused work, not a rebuild.

The honest risk is not that any of this is difficult. It is that a system which demonstrates well gets adopted before items 1 to 4 are addressed, and the gap is then discovered on the day it costs something.